# Software & shipping

## The simple version

A workshop for managers and emerging leaders
120 minutes • No coding experience required

# What you will leave with

Explain frontend, backend, APIs and databases using one example.

Trace a request and its response.

Create a commit and a pull request using GitHub Desktop.

Distinguish environments, deployment and release.

Ask for evidence before a launch.

# Our 120-minute route

00–10 Shared language and opening discussion

10–30 Software parts, requests and responses

30–40 Activity: act out a leave request

40–55 Git and GitHub essentials

55–90 GitHub Desktop lab

90–110 Environments and a deployment demonstration

110–120 Release decision and exit check

# One employee leave request



An employee requests two days off.

The product checks the rules and records the request.

The employee sees “Pending approval”.

# Four parts of the application

Frontend: the form and status the employee sees.

Backend: code that checks identity, balance and policy.

API: the agreed way software asks for actions or data.

Database: stored records of employees and requests.

# Frontend and backend responsibilities

## Frontend

Displays dates and leave balance.

Helps users enter valid information.

Shows success, errors and loading states.

## Backend

Checks who may request or approve leave.

Enforces rules even if the screen is bypassed.

Coordinates data reads and writes.

# A request asks. A response answers.

## Request

POST /leave-requests

Body: { "days": 2 }

Meaning: create a leave request.

## Response

201 Created

Body: { "id": "LR-104", "status": "pending" }

Meaning: recorded, awaiting approval.

# A database remembers the business state

A record might contain request ID, employee ID, dates and status.

A query reads matching records, such as my pending requests.

A write adds or changes a record.

A schema defines the structure and rules for stored data.

**A code rollback does not automatically undo data changes.**

# When the request fails

400: the request is invalid. Explain what the user should fix.

403: the user lacks permission. Check the access rule.

500: the server failed. Investigate logs and impact.

Timeout: the result is uncertain. Check whether the write happened.

**Repeated clicks can create duplicate work unless the system handles retries.**

# Activity 1: become the application

Teams of five: employee, frontend, API messenger, backend, database.

Pass a paper request for two days through the team.

The backend checks a fictional balance of five days.

The database records “pending”. Return the result to the employee.

Repeat with eight days. Explain the different response.

**10 minutes • Deliverable: a labeled request journey**

# Git, GitHub and GitHub Desktop

## On your computer

Git tracks version history.

GitHub Desktop gives Git a graphical interface.

A local repository holds files and history.

## Shared online

GitHub hosts repositories.

Pull requests support discussion and review.

Access rules control who can read or change work.

# The vocabulary of a change

Repository: project files plus version history.

Commit: a recorded snapshot with a message.

Branch: a separate line of work within the repository.

Pull request (PR): a proposal to review and merge changes.

Merge: combine changes into the target branch.

# Five actions with different results

Save: update a file on your computer.

Commit: record selected changes in local history.

Push: send commits to the remote repository.

Merge: integrate a branch into another branch.

Deploy: put a software version into an environment.

**Automation can connect these actions. The actions still mean different things.**

# Review makes a change understandable

What user problem does this change solve?

Which files and behavior changed?

What evidence shows it works?

What risk remains, and how could we recover?

**A useful PR explains the reason and the result.**

# Lab setup checkpoint

GitHub account with verified email and working sign-in.

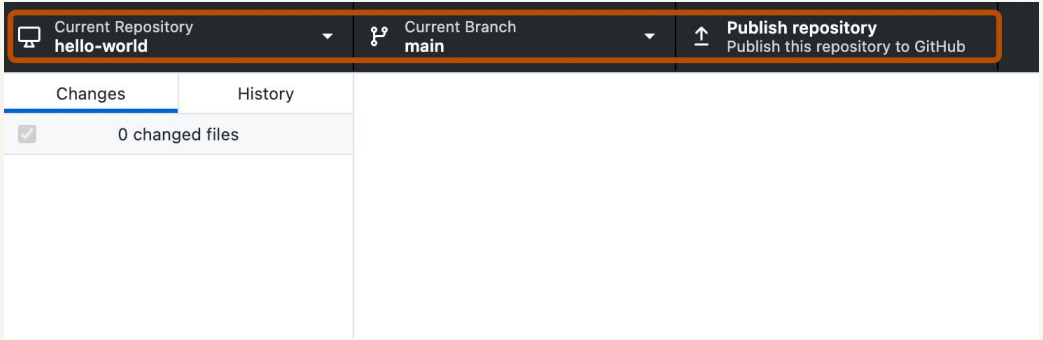
GitHub Desktop on supported Windows or macOS.

A browser and a plain-text editor, such as VS Code.

The supplied starter files saved on your laptop.

Use fictional content in a personal training repository.

# Your first repository



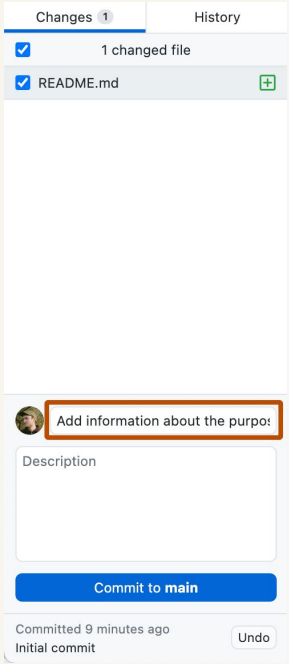

Create a local repository named shipping-lab.

Initialize a README.

Confirm the current repository and branch.

GitHub Docs screenshot • UI may vary

# Your first manual commit




Copy index.html and README.md from the starter folder into the repo.

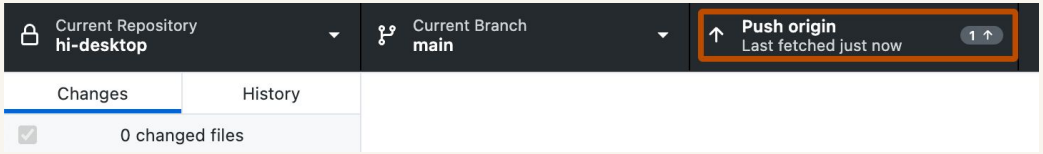
Review the Changes list.

Summary: Add leave request demo.

Commit to main. Confirm it in History.

GitHub Docs screenshot • UI may vary

# Publishing and pushing




Publish repository and keep it private.

Open View on GitHub.

Confirm your files and commit message online.

Later commits upload with Push origin.

GitHub Docs screenshot • UI may vary

# A branch and a pull request

Create clarify-button from main.

Change “Submit” to “Request leave” in index.html.

Save, inspect the diff and commit: Clarify leave request button.

Publish branch. Preview Pull Request with base main.

Create the PR in your browser. Explain why and how you checked it.

# The lab completion check

A partner can explain the diff and the purpose of the PR.

Merge the exercise PR after review.

Switch to main in Desktop, Fetch origin, then Pull origin if offered.

Refresh the local page and confirm “Request leave”.

Point to the commit, branch and PR in your own project.

# Environments are places software runs

## Local and development

Local: on a developer’s computer.

Development: a shared place to try changes.

Usually uses test data and frequent updates.

## Staging and production

Staging: a rehearsal for release.

Production: the real user service.

Config, access and data may differ.

# Deployment and release

Build: prepare software files or a runnable package.

Deploy: install or activate that version in an environment.

Release: make a feature available to its intended users.

Observe: verify user journeys and watch errors after the change.

**A feature flag can separate deployment from release.**

# Demonstration: a page becomes a website

Local: open index.html and notice the file address.

Hosted: open the prepared GitHub Pages URL.

Change Version 1 to Version 2, commit and push.

Watch the deployment run, then verify Version 2 at the URL.

**This demonstration hosts a static frontend. It has no backend or database.**

# Evidence for a release decision

Scope: which users and workflows change?

Readiness: what passed, and what remains untested?

Ownership: who watches the launch and responds?

Recovery: can we disable or roll back safely?

Success: which user behavior or error signal will we check?

# Activity 2: ship or wait?

The leave request screen passed in staging.

Production approvals return errors for some managers.

The team has no tested recovery path.

A sponsor wants the rollout expanded today.

**In pairs: decide, name the missing evidence, and assign the next action.**

# Your exit check

Which component should enforce the leave policy?

What travels in a request and in a response?

Does a commit appear on GitHub before a push?

How is a branch different from staging?

What evidence would you request before release?

**Take away: your repository, your PR, and one better leadership question.**

# Reference links and continued practice

GitHub Docs: Git, Desktop commits, branches and pull requests.

MDN: web applications, HTTP requests and responses.

GitHub Pages: publishing configuration.

Next practice: make a second small change and ask a peer to review it.